Pandas Cheat Sheet

`import pandas as pd` · Data Engineer Interview Edition · `df = pd.DataFrame(...)`

I/O & INSPECTION

FUNCTION	FORMAT	EXAMPLES

read_csv / to_csv
Read/write delimited files
	`pd.read_csv(path, sep=',', parse_dates=[cols], dtype={}, usecols=[])`
	`df.to_csv(path, index=False)`
		`df = pd.read_csv('orders.csv', parse_dates=['order_date'], dtype={'id': str})`
		`df.to_csv('out.csv', index=False)`
		✓ always index=False when writing

info / dtypes / describe
Schema + stats snapshot
	`df.info()`
	`df.dtypes`
	`df.describe(include='all')`
		`df.info()  # nulls + dtypes in one shot`
		`df.describe(include='all')  # numeric + object cols`

head / tail / sample Quick peek	`df.head(n)` `df.sample(n, random_state=42)`	`df.sample(5, random_state=42)`
value_counts Frequency table	`df['col'].value_counts(dropna=False, normalize=False)`	`df['status'].value_counts(dropna=False)` `df['status'].value_counts(normalize=True)  # proportions`

CLEANING & NULLS

FUNCTION	FORMAT	EXAMPLES

dropna
Remove rows/cols with nulls
	`df.dropna(axis=0|1, subset=[cols], how='any'|'all', thresh=n)`	`df.dropna()  # drop any row with NaN`
		`df.dropna(subset=['email', 'phone'])`
		`df.dropna(thresh=3)  # keep rows with ≥3 non-NaN`
		`df.dropna(axis=1)  # drop columns instead`
		⚠ add inplace=True or reassign

fillna
Fill missing values
	`df.fillna(value, method='ffill'|'bfill')`
	`df['col'].fillna(df['col'].mean())`
		`df.fillna(0)  # fill all NaN with 0`
		`df['salary'].fillna(df['salary'].median())`
		`df.fillna(method='ffill')  # forward fill (time series)`
		`df.fillna({'city': 'Unknown', 'age': 0})`

FUNCTION	FORMAT	EXAMPLES

drop_duplicates
Remove duplicate rows
	`df.drop_duplicates(subset=[cols], keep='first'|'last'|False)`	`df.drop_duplicates()  # all cols match`
		`df.drop_duplicates(subset=['user_id'], keep='last')`
		`df.drop_duplicates(subset=['id'], keep=False)  # drop ALL dupes`

isna / notna
Null masks
	`df.isna().sum()`
	`df[df['col'].isna()]`
		`df.isna().sum()  # null count per column`
		`df[df['revenue'].isna()]  # filter null rows`
		`df.isna().sum() / len(df) * 100  # null % per col`

SHAPING & TRANSFORMING

FUNCTION	FORMAT	EXAMPLES

rename
Rename columns/index
	`df.rename(columns={'old': 'new'})`
	`df.reset_index().rename(columns={'index': 'name'})`
		`df.rename(columns={'cust_id': 'customer_id'})`
		`# rename groupby result index to column:`
		`df.groupby('dept')['sal'].mean().reset_index().rename(columns={'sal': 'avg_sal'})`
		`# rename all cols at once:`
		`df.columns = [c.lower().replace(' ', '_') for c in df.columns]`

reset_index Move index → column	`df.reset_index(drop=False)`	`result = df.groupby('city').size().reset_index(name='count')` `df.reset_index(drop=True)  # discard old index`

sort_values
Sort rows by columns
	`df.sort_values(by=[cols], ascending=True|False, na_position='last')`	`df.sort_values('salary', ascending=False)`
		`df.sort_values(['dept', 'salary'], ascending=[True, False])`
		`df.sort_values('score', na_position='last')`

drop Drop rows or columns	`df.drop(columns=[cols])` `df.drop(index=[labels])`	`df.drop(columns=['temp_col', 'junk'])` `df.drop(index=df[df['flag'] == 1].index)`

assign
Add/overwrite columns
	`df.assign(new_col=expr_or_lambda)`	`df.assign(tax=df['price'] * 0.18)`
		`df.assign(full_name=lambda x: x['first'] + ' ' + x['last'])`
		`✓ chainable — returns new df`

apply Row/col-wise function	`df['col'].apply(func)` `df.apply(func, axis=0|1)`	`df['name'].apply(str.title)` `df['score'].apply(lambda x: 'pass' if x ≥ 50 else 'fail')`

FUNCTION	FORMAT	EXAMPLES
		⚠ slow on large data — prefer vectorized ops

astype
Cast column types
	`df['col'].astype(dtype)`
	`df.astype({'col1': dtype, 'col2': dtype})`
		`df['age'].astype(int)`
		`df['id'].astype(str)`
		`df.astype({'price': float, 'qty': int})`
		`df['cat'].astype('category')  # saves memory`
		⚠ NaNs in int col → use pd.Int64Dtype() or float first

where / np.where Conditional values	`np.where(condition, val_if_true, val_if_false)` `pd.cut / pd.qcut for binning`	`df['tier'] = np.where(df['spend'] > 1000, 'gold', 'silver')` `df['band'] = pd.cut(df['age'], bins=[0,18,35,60,99], labels=['teen','young','mid','senior'])`

FILTERING & SELECTION

FUNCTION	FORMAT	EXAMPLES

isin
Filter by value list
	`df[df['col'].isin([val1, val2])]`
	`df[~df['col'].isin([...])]  # NOT in`
		`df[df['country'].isin(['IN', 'US', 'UK'])]`
		`df[~df['status'].isin(['cancelled', 'refunded'])]`
		`# filter using another df's column:`
		`df[df['id'].isin(df2['id'])]`

query SQL-like filter syntax	`df.query("col > val and col2 == 'str'")`	`df.query("age > 25 and dept == 'eng'")` `threshold = 1000; df.query("revenue > @threshold")  # use @ for vars`
loc / iloc Label / position select	`df.loc[row_label, col_label]` `df.iloc[row_int, col_int]`	`df.loc[df['score'] > 90, ['name', 'score']]` `df.iloc[0:5, 2:5]  # first 5 rows, cols 2-4`
nlargest / nsmallest Top/bottom N rows	`df.nlargest(n, 'col')` `df.nsmallest(n, 'col')`	`df.nlargest(10, 'revenue')` `df.nlargest(5, ['revenue', 'qty'])`

MERGING & COMBINING

FUNCTION	FORMAT	EXAMPLES

merge
SQL-style joins
	`pd.merge(left, right, on='col', how='inner'|'left'|'right'|'outer', suffixes=('_x','_y'))`	`# inner join (default):`
		`pd.merge(orders, customers, on='cust_id')`
		`# left join, different key names:`

FUNCTION	FORMAT	EXAMPLES

		`pd.merge(df1, df2, left_on='id', right_on='user_id', how='left')`
		
		`# anti-join (rows in left NOT in right):`
		`pd.merge(df1, df2, on='id', how='left', indicator=True).query("_merge == 'left_only'")`
		
		✓ indicator=True → _merge col shows 'left_only','both','right_only'

concat
Stack dfs vertically or horizontally
	`pd.concat([df1, df2], axis=0|1, ignore_index=True, keys=['a','b'])`	`# stack rows (union):`
		`pd.concat([jan, feb, mar], ignore_index=True)`
		
		`# stack columns side by side:`
		`pd.concat([df1, df2], axis=1)`
		
		`# keep track of origin:`
		`pd.concat([df1, df2], keys=['2024', '2025'])`
		
		⚠ axis=0 needs same columns; mismatches → NaN

join Index-based join	`df1.join(df2, on='col', how='left'|'inner')`	`df1.set_index('id').join(df2.set_index('id'), how='inner')`

GROUPBY & AGGREGATION

FUNCTION	FORMAT	EXAMPLES

groupby + agg
Multi-function aggregation
	`df.groupby('col').agg({'col2': ['sum','mean'], 'col3': 'count'})`
	`df.groupby('col').agg(alias=('col2', 'sum'))`
		`df.groupby('dept')['salary'].agg(['mean', 'max', 'count'])`
		
		`# named aggregation (clean output):`
		`df.groupby('dept').agg(`
		`  avg_sal=('salary', 'mean'),`
		`  headcount=('emp_id', 'nunique')`
		`).reset_index()`
		
		✓ named agg avoids MultiIndex columns

groupby + transform
Broadcast agg back to df
	`df.groupby('col')['col2'].transform('mean'|func)`	`# add dept avg as new column (same row count):`
		`df['dept_avg_sal'] = df.groupby('dept')['salary'].transform('mean')`
		
		`# pct of group total:`
		`df['pct'] = df['sales'] / df.groupby('region')['sales'].transform('sum')`
		
		✓ key difference from agg: same shape as input

pivot_table 2D cross-tab aggregation	`df.pivot_table(values, index, columns, aggfunc, fill_value)`	`df.pivot_table(values='sales', index='region', columns='month', aggfunc='sum', fill_value=0)`
rank / cumsum Window-style calcs	`df.groupby('col')['col2'].rank(method='dense', ascending=False)` `df['col'].cumsum()`	`# rank within group (dense = no gaps):` `df['rank'] = df.groupby('dept')['salary'].rank(method='dense', ascending=False)`

FUNCTION	FORMAT	EXAMPLES
		`df['running_total'] = df.groupby('user')['spend'].cumsum()`

STRING OPERATIONS

FUNCTION	FORMAT	EXAMPLES

str.contains
Regex / substring filter
	`df['col'].str.contains('pattern', case=True, na=False, regex=True)`	`df[df['email'].str.contains('@gmail', na=False)]`
		`df[df['name'].str.contains('kumar|sharma', case=False, na=False)]`
		⚠ always set na=False to avoid NaN propagation

str.replace / strip / split
Clean text
	`df['col'].str.replace('old','new', regex=True)`
	`df['col'].str.strip()`
	`df['col'].str.split(',', expand=True)`
		`df['phone'].str.replace(r'[^0-9]', '', regex=True)  # digits only`
		`df['city'].str.strip().str.lower()`
		`df[['first', 'last']] = df['name'].str.split(' ', expand=True)`

str.extract / extractall Regex capture groups	`df['col'].str.extract(r'(pattern)')`	`df['pin'] = df['address'].str.extract(r'(\d{6})')` `df['domain'] = df['email'].str.extract(r'@(.+)')`
str.len / startswith / endswith Quick string checks	`df['col'].str.len()` `df['col'].str.startswith('prefix')`	`df[df['code'].str.startswith('INV')]` `df[df['name'].str.len() > 50]  # suspiciously long names`

DATETIME OPERATIONS

FUNCTION	FORMAT	EXAMPLES

to_datetime
Parse strings → datetime
	`pd.to_datetime(df['col'], format='%Y-%m-%d', errors='coerce')`	`pd.to_datetime(df['order_date'], format='%d-%m-%Y')`
		`pd.to_datetime(df['ts'], unit='s')  # unix timestamp`
		`pd.to_datetime(df['date'], errors='coerce')  # bad dates → NaT`

dt.strftime / dt.components
Extract date parts
	`df['col'].dt.year / .month / .day / .hour`
	`df['col'].dt.strftime('%Y-%m')`
	`df['col'].dt.day_name()`
		`df['year'] = df['date'].dt.year`
		`df['month_str'] = df['date'].dt.strftime('%Y-%m')  # '2024-03'`
		`df['weekday'] = df['date'].dt.day_name()  # 'Monday'`
		`df['quarter'] = df['date'].dt.quarter`

dt.date / dt.floor / resample	`df['col'].dt.date  # strip time` `df['col'].dt.floor('D'|'H'|'min')`	`df['hour_bucket'] = df['ts'].dt.floor('H')`

FUNCTION	FORMAT	EXAMPLES
Truncate & resample	`df.resample('M', on='date')['val'].sum()`	`df.resample('M', on='order_date')['revenue'].sum().reset_index()`
timedelta / date_range Date arithmetic	`df['col2'] - df['col1']  → Timedelta` `pd.date_range(start, end, freq='D')`	`df['days_open'] = (df['close_date'] - df['open_date']).dt.days` `pd.date_range('2024-01-01', '2024-12-31', freq='M')`

PERFORMANCE & MEMORY

FUNCTION	FORMAT	EXAMPLES

memory_usage
Diagnose memory bloat
	`df.memory_usage(deep=True).sum()`	`df.memory_usage(deep=True).sum() / 1e6  # MB`
		`# reduce: cast objects → category, float64 → float32`
		`df['status'] = df['status'].astype('category')`

pipe
Chain custom functions cleanly
	`df.pipe(func1).pipe(func2, arg)`	`(df`
		`  .pipe(clean_nulls)`
		`  .pipe(normalize_dates)`
		`  .pipe(add_features))`
		✓ preferred pattern in production pipelines

explode Unnest list columns	`df.explode('col')`	`# df['tags'] has lists like ['a','b','c']` `df.explode('tags').reset_index(drop=True)`
melt Wide → long (unpivot)	`df.melt(id_vars=[cols], value_vars=[cols], var_name, value_name)`	`df.melt(id_vars=['id', 'name'], value_vars=['q1','q2','q3'], var_name='quarter', value_name='sales')`
crosstab Frequency cross-table	`pd.crosstab(df['col1'], df['col2'], normalize=False)`	`pd.crosstab(df['gender'], df['dept'], normalize='index')  # row %`

import pandas as pd | import numpy as np ⚡ vectorized ops > apply > iterrows (never in prod) 🔑 always reset_index() after groupby for clean columns 📌 na=False in str ops · errors='coerce' in to_datetime